

Hybrid MPI+OpenMP Performance of PETSc Krylov Solvers on ARCHER2

Leyan Li

MSc in High Performance Computing
EPCC, University of Edinburgh

August 2026

A dissertation submitted in partial fulfilment
of the requirements for the degree of
Master of Science.

Abstract

Acknowledgements

Contents

Abstract	i
Acknowledgements	ii
1 Introduction	1
1.1 Motivation	1
1.2 Aims and Research Questions	1
1.3 Contributions	1
1.4 Dissertation Structure	1
2 Background and Related Work	2
2.1 Parallel Programming Models	2
2.2 Limitations of Flat MPI	2
2.3 Hybrid MPI+OpenMP	2
2.4 ARCHER2 Node Architecture and NUMA	2
2.5 PETSc and Krylov Solvers	2
2.6 Performance Analysis Tools	2
2.7 Summary and Research Gap	2
3 Experimental Platform and Methodology	3
3.1 Hardware and Software Environment	3
3.2 Benchmark Problems and Drivers	3
3.3 Solver Configuration	3
3.4 Configuration Space and Selection Rules	3
3.5 Job Configuration and Thread Placement	4
3.6 Metrics	4
3.7 Reproducibility	4
4 Single-Node Performance Evaluation	5
4.1 Full Rank–Thread Sweep	5
4.2 Rationale for the Reduced Configuration Set	5
4.3 Effect of Solver Choice	5
4.4 Single-Node Summary	5
5 Multi-Node Scaling (2D and 3D)	6
5.1 Experimental Setup Recap	6
5.2 2D Results	6
5.3 3D Results	6
5.4 Cross-Dimension Comparison	6
5.5 Interpretation	6

6	Profiling and Bottleneck Analysis	7
6.1	Why a Repeated-Solve Driver	7
6.2	Analysis Methodology	7
6.3	MAP Profiling Matrix Results	7
6.4	Bottleneck Identification	7
6.5	Preconditioner Reuse in the Scaling Context	7
7	Discussion	8
7.1	Answering the Research Questions	8
7.2	Threats to Validity	8
7.3	Practical Guidance for PETSc on ARCHER2	8
8	Conclusion and Future Work	9
8.1	Conclusions	9
8.2	Future Work	9
A	Reproducibility: Repository and Scripts	10
B	Full log_view Excerpts	11

Chapter 1

Introduction

1.1 Motivation

1.2 Aims and Research Questions

This study measures PETSc rank–thread configurations for a Poisson problem on ARCHER2: flat MPI against hybrid MPI+OpenMP, in 2D and 3D, at four problem sizes, from 1 to 64 nodes. It asks whether hybrid execution reduces runtime, which configuration gives the lowest runtime, and where the time goes.

1.3 Contributions

1.4 Dissertation Structure

Chapter 2

Background and Related Work

2.1 Parallel Programming Models

2.2 Limitations of Flat MPI

2.3 Hybrid MPI+OpenMP

2.4 ARCHER2 Node Architecture and NUMA

2.5 PETSc and Krylov Solvers

2.6 Performance Analysis Tools

2.7 Summary and Research Gap

Chapter 3

Experimental Platform and Methodology

3.1 Hardware and Software Environment

All experiments ran on ARCHER2. Each node has two AMD EPYC 7742 processors, 128 physical cores, and eight NUMA regions of 16 cores each. The software stack was PrgEnv-gnu 8.4.0, gcc 11.2.0, and cray-mpich 8.1.27. PETSc 3.24.4 was built from source as `arch-omp-opt` with `--with-openmp=1 --with-openmp-kernels=1` and `-O3`. One thread per rank is used as the MPI-only baseline.

3.2 Benchmark Problems and Drivers

The 2D benchmark is PETSc `ex2`, a five-point stencil discretisation of the Laplace operator on an $m \times n$ grid. The 3D benchmark is a driver written for this project, a seven-point stencil on an $m \times n \times p$ grid; it was added because the surface-to-volume ratio differs from 2D. Profiling uses `ex2_repeat` (Chapter 6).

3.3 Solver Configuration

The default `ex2` solver is GMRES with block Jacobi. At 4500×4500 and above, the error norm reached order 10^{10} , so the iteration did not converge within the default limit. The solver was changed rather than the tolerance relaxed. Production runs use

```
-ksp_type cg -pc_type gamg -ksp_rtol 1e-5 -ksp_atol 1e-10
```

CG applies because the discretised Laplacian is symmetric positive definite. GAMG is used because multigrid convergence is close to mesh-independent for elliptic problems. Across 5.0M to 41.0M unknowns the solver takes 11–12 iterations with an error norm of order 10^{-2} (0.026 at 4500×4500). Constant iteration counts mean runtime differences reflect per-iteration and setup cost, not iteration count.

GAMG cost has two phases: setup (`PCSetUp`, building the coarse hierarchy) and solve (`KSPSolve`). In a single-solve run, setup is about 45% of runtime; in applications the preconditioner is reused across solves. Chapters 4 and 5 report total time including setup; Chapter 6 isolates the solve phase.

3.4 Configuration Space and Selection Rules

Three rules reduce the space.

Threads per rank are restricted to $\{1, 2, 4, 8\}$. A single-node sweep showed 16 or more threads per rank were slower at every core count, and a single rank (pure OpenMP) reduced runtime by under 20% from 1 to 128 threads. At eight threads or fewer a rank stays within one or two NUMA regions.

Nodes are fully populated: $\text{ppn} \times \text{threads} = 128$, giving four layouts 128×1 , 64×2 , 32×4 , and 16×8 . Same node count means same core count, so a comparison isolates rank–thread balance from core count.

A configuration at size N on C cores is included only if $10^4 < N/C < 10^6$. This sets the valid node counts per size. Four sizes are used: 5.0M, 10.0M, 20.25M, and 41.0M. The 2D and 3D grid dimensions are matched per nominal size (for example $4500^2 \approx 273^3 \approx 20.25 \text{ M}$).

3.5 Job Configuration and Thread Placement

Each rank and its threads are pinned to a contiguous set of cores with:

```
OMP_PLACES=cores    OMP_PROC_BIND=close
srun --hint=nomultithread --distribution=block:block --exact
```

3.6 Metrics

Runtime is the maximum `Time (sec)` over ranks from `-log_view`, taken as the median over repetitions. Scaling is computed per layout against that layout’s smallest node count (node-relative, not single-core; this differs from the feasibility study). For a layout ℓ at n nodes,

$$S_\ell(n) = \frac{T_\ell(n_0)}{T_\ell(n)}, \quad E_\ell(n) = \frac{S_\ell(n)}{n/n_0},$$

where n_0 is the smallest node count run for that problem size.

3.7 Reproducibility

Chapter 4

Single-Node Performance Evaluation

4.1 Full Rank–Thread Sweep

With one MPI rank and threads from 1 to 128 at one million unknowns, runtime went from 246 s to 208 s (under 20%) while cores increased 128-fold. At 32 cores and one million unknowns, 8×4 ran in 23.0 s and 32×1 in 29.8 s; at 16 cores, 4×4 and 8×2 were each more than twice as fast as 16×1 . Configurations with 16 or more threads per rank were slower at every core count tested.

4.2 Rationale for the Reduced Configuration Set

These results set the reduced set used in the rest of the study: threads per rank in $\{1, 2, 4, 8\}$ on fully populated nodes.

4.3 Effect of Solver Choice

The single-node sweep was first run with the default solver. Under CG+GAMG the layout order changes. At 5.0M unknowns on one node, the default solver is fastest at 128×1 (64.0 s, against 76.0 s for 32×4); under CG+GAMG the fastest is 32×4 (12.3 s, against 13.8 s for 128×1).

4.4 Single-Node Summary

Chapter 5

Multi-Node Scaling (2D and 3D)

5.1 Experimental Setup Recap

The four layouts run at four sizes in 2D and 3D over the node counts admitted by the unknowns-per-core window: up to 8 nodes at 5.0M, 16 at 10.0M, 32 at 20.25M, and 64 at 41.0M. All runs use CG+GAMG; iteration counts stay at 11–12 across sizes and node counts.

5.2 2D Results

32×4 had the lowest median runtime at every size and at almost every node count. For 41.0M unknowns it was faster than 128×1 by 11.3% on one node (105.0s against 118.4s), 11.7% on 16 nodes, and 12.0% on 64 nodes. 64×2 was a few percent behind 32×4 .

From one node to two, speedup was 0.93–0.99 for every layout and size. From two nodes on, each doubling gave speedups near 2.0 (2.00, 1.99, 2.00, 2.01 for 32×4 at 41.0M). Efficiency relative to one node settled at 45–48%.

At scale, 16×8 reached 33% efficiency at 20.25M on 32 nodes (32×4 : 48%) and 18% at 41.0M on 64 nodes, with runtime 9.19s against 4.03s for 32×4 .

5.3 3D Results

The layout order matched 2D (32×4 fastest, 128×1 slowest of the four, 16×8 weakest at scale), with two differences. From one node to two, two nodes were faster than one (speedup 1.02–1.10); efficiency at two nodes was 51–55% (2D: 47–49%). The 3D seven-point stencil exchanges a larger halo relative to subdomain interior than the 2D five-point stencil. At 41.0M on 64 nodes, 16×8 kept 35% efficiency (2D: 18%) and was 11% slower than 32×4 (2D: 128% slower).

5.4 Cross-Dimension Comparison

5.5 Interpretation

Chapter 6

Profiling and Bottleneck Analysis

6.1 Why a Repeated-Solve Driver

In a single solve at 20.25M with 16×8 , `-log_view` gave `PCSetUp` 45% of time and `KSPSolve` 10%. `ex2_repeat` runs the solve inside a `RepeatedSolves` log stage, zeroes the initial guess before each solve, and reuses the preconditioner (`-ksp_reuse_preconditioner` and `-pc_gamg_reuse_interpolation true`). With 20 solves the stage held 69% of time and 93% of flops; within the stage `KSPSolve` was 100% of time; each of the 19 repeats took 11 iterations.

6.2 Analysis Methodology

The method combines MAP 25.0.4 timelines with `-log_view %T` (share of stage time) and `%F` (share of stage flops). An event whose `%T` exceeds its `%F` spends the difference on communication, synchronisation, or memory traffic. Table 6.1 lists single-point data for the `RepeatedSolves` stage at 20.25M with 16×8 .

Table 6.1: Per-event time and flop shares, `RepeatedSolves` stage, 20.25M, 16×8 .

Event	%T	%F	GF/s
<code>MatMult</code>	39	55	30.5
<code>MatMultTranspose</code>	18	6	6.7
<code>VecTDot+VecNorm</code>	8	6	—
<code>PCApply</code> (agg.)	82	82	21.6

`MatMultTranspose` (GAMG restriction) has a `%T` three times its `%F` and a rate $4.6\times$ below `MatMult`. `VecTDot` and `VecNorm` are the CG global reductions.

6.3 MAP Profiling Matrix Results

6.4 Bottleneck Identification

6.5 Preconditioner Reuse in the Scaling Context

Chapter 7

Discussion

7.1 Answering the Research Questions

7.2 Threats to Validity

7.3 Practical Guidance for PETSc on ARCHER2

Chapter 8

Conclusion and Future Work

8.1 Conclusions

8.2 Future Work

Appendix A

Reproducibility: Repository and Scripts

Appendix B

Full log_view Excerpts